

Machine learning for vision

Winter 2016

Roland Memisevic

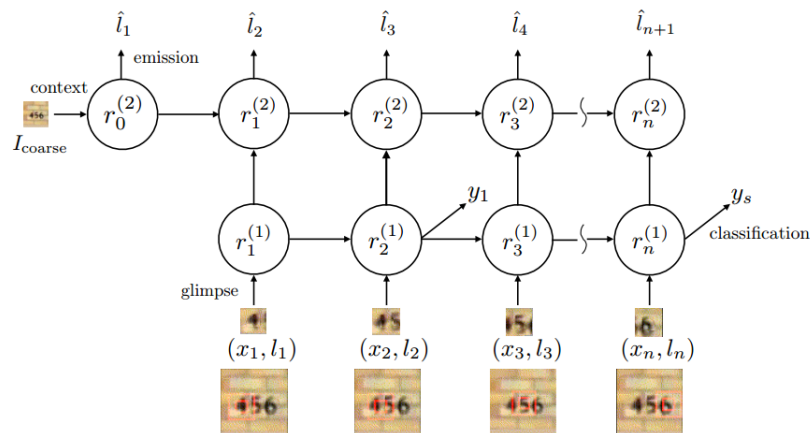
Lecture 7, March 22, 2016

Navigation icons

Roland Memisevic

Machine learning for vision

RL in vision: Attention



- ▶ (Ba, Mnih, Kavukcuoglu; 2014)

Navigation icons

Roland Memisevic

Machine learning for vision

Reinforcement learning

- ▶ The common (supervised) learning paradigm: you have a dataset of pairs (x, y)
- ▶ Not all (hardly any?) real-world application scenarios fit this paradigm
- ▶ A more general paradigm has an agent interact with an environment (or simulator) which determines the value of the networks actions on-the-fly
- ▶ Characteristics of this paradigm:
 - ▶ time-series
 - ▶ delayed rewards
 - ▶ exploration-exploitation
 - ▶ the ultimate goal is a *policy*

Navigation icons

Roland Memisevic

Machine learning for vision

Policy search (“Actor”)

- ▶ Policy:

$$\pi_{\theta}(\mathbf{a}_{0:T}|\mathbf{h}_{0:T}) = \prod_{t=0}^T \pi_{\theta}(\mathbf{a}_t|\mathbf{h}_{0:t})$$

- ▶ Reward:

$$\begin{aligned} J^{\pi} &= \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T r_t(\mathbf{a}_t) \right] \\ &= \sum_{\mathbf{a}_{0:T}} \left[\sum_t r_t(\mathbf{a}_t) \right] \pi_{\theta}(\mathbf{a}_{0:T}|\mathbf{h}_{0:T}) \end{aligned}$$

see <https://www.youtube.com/watch?v=kUiR0RLmGCo>

Navigation icons

Roland Memisevic

Machine learning for vision

Policy gradient

$$\begin{aligned}
 \nabla_{\theta} J^{\pi} &= \nabla_{\theta} \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T r_t(\mathbf{a}_t) \right] \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T}) \\
 &= \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T r_t(\mathbf{a}_t) \right] \nabla_{\theta} \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T}) \\
 &= \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T r_t(\mathbf{a}_t) \right] \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T}) \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T}) \\
 &= \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T r_t(\mathbf{a}_t) \right] \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) \right] \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T}) \\
 &= \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) \sum_{n=t}^T r_n(\mathbf{a}_n) \right] \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T})
 \end{aligned}$$

Roland Memisevic

Machine learning for vision

Variance reduction

- ▶ The sample approximation is correct in expectation but it can have high variance.
- ▶ It is widely agreed that variance reduction techniques are necessary to make learning tractable.
- ▶ A common way to reduce variance is based on the observation:

$$\begin{aligned}
 \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) \right] &= \mathbb{E}_{\pi} \left[\frac{\nabla_{\theta} \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t})}{\pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t})} \right] \\
 &= \nabla_{\theta} \mathbb{E}_{\pi} [1] \\
 &= 0
 \end{aligned}$$

Navigation icons

Roland Memisevic

Machine learning for vision

Sampling

$$\nabla_{\theta} J^{\pi} = \sum_{\mathbf{a}_{0:T}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) \sum_{n=t}^T r_n(\mathbf{a}_n) \right] \pi_{\theta}(\mathbf{a}_{0:T} | \mathbf{h}_{0:T})$$

- ▶ Sample trajectories according to $\mathbf{a}_{0:T}^{(i)} \sim \pi(\mathbf{a}_{0:T} | \mathbf{h}_{0:T})$
- ▶ and optimize the sample approximation:

$$\widehat{\nabla_{\theta} J^{\pi}} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{h}_{0:t}) \sum_{n=t}^T r_n(\mathbf{a}_n^{(i)})$$

Navigation icons

Roland Memisevic

Machine learning for vision

Variance reduction

- ▶ To reduce clutter, define $R_t = \sum_{n=t}^T r_n(\mathbf{a}_n)$
- ▶ Introduce constant c (or $c(\mathbf{h}_{0:t})$)
- ▶ A lower-variance estimate is given by:

$$\begin{aligned}
 &\mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) (R_t - c) \right] \\
 &= \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) R_t \right] - c \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) \right] \\
 &= \mathbb{E}_{\pi} \left[\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{h}_{0:t}) R_t \right]
 \end{aligned}$$

- ▶ c is often called *baseline* and can be learned to represent (or predict) the expected reward.
- ▶ This allows us to directly exploit what is predictable about the rewards.

Navigation icons

Roland Memisevic

Machine learning for vision

DQN



- (Mnih et al, 2013)

◀ ▶ ⏪ ⏩ 🔍 🔄

Roland Memisevic

Machine learning for vision

Dynamic programming

$$\begin{aligned}
 V^*(\mathbf{s}_0) &= \max_{\pi} \mathbb{E}_{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \dots} \left[\sum_{t=0}^{\infty} \gamma^t r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_0 \right] \\
 &= \max_{\pi} \mathbb{E}_{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \dots} \left[r_0(\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1) + \sum_{t=1}^{\infty} \gamma^t r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_0 \right] \\
 &= \max_{\mathbf{a}_0} \mathbb{E}_{\mathbf{s}_1} \left[r_0(\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1) \right. \\
 &\quad \left. + \max_{\pi} \mathbb{E}_{\mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_4, \dots} \left[\sum_{t=1}^{\infty} \gamma^t r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_1 \right] \middle| \mathbf{s}_0 \right] \\
 &= \max_{\mathbf{a}_0} \mathbb{E}_{\mathbf{s}_1} \left[r_0(\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1) \right. \\
 &\quad \left. + \gamma \max_{\pi} \mathbb{E}_{\mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_4, \dots} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_1 \right] \middle| \mathbf{s}_0 \right] \\
 &= \max_{\mathbf{a}_0} \mathbb{E}_{\mathbf{s}_1} \left[r_0(\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1) + \gamma V^*(\mathbf{s}_1) \right]
 \end{aligned}$$

◀ ▶ ⏪ ⏩ 🔍 🔄

Roland Memisevic

Machine learning for vision

Dynamic programming and TD (“Critic”)

- Policy as a function of state (not an RNN):

$$\mathbf{a}_t = \pi(\mathbf{s}_t)$$

- Value function (depends on π):

$$V^{\pi}(\mathbf{s}_0) = \mathbb{E}_{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \dots} \left[\sum_{t=0}^{\infty} \gamma^t r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_0 \right]$$

- Optimal value function:

$$V^*(\mathbf{s}_0) = \max_{\pi} \mathbb{E}_{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \dots} \left[\sum_{t=0}^{\infty} \gamma^t r_t(\mathbf{s}_t, \pi(\mathbf{s}_t), \mathbf{s}_{t+1}) \middle| \mathbf{s}_0 \right]$$

◀ ▶ ⏪ ⏩ 🔍 🔄

Roland Memisevic

Machine learning for vision

Bellman’s equation

$$V^*(\mathbf{s}) = \max_{\mathbf{a}} \mathbb{E}_{\mathbf{s}'} \left[r(\mathbf{s}, \mathbf{a}, \mathbf{s}') + \gamma V^*(\mathbf{s}') \middle| \mathbf{s} \right]$$

- TD learning: Draw $\tilde{\mathbf{s}}'$ randomly and set

$$V_{t+1}^{\pi}(\mathbf{s}) = V_t^{\pi}(\mathbf{s}) + \eta \left(r(\mathbf{s}, \mathbf{a}, \tilde{\mathbf{s}}') + \gamma V_t^{\pi}(\tilde{\mathbf{s}}') - V_t^{\pi}(\mathbf{s}) \right)$$

◀ ▶ ⏪ ⏩ 🔍 🔄

Roland Memisevic

Machine learning for vision

